ACADEMIC PROJECT REPORT | DATA WAREHOUSING & MINING (DWM)

Urban Air Quality & Pollution Forecasting System

A Unified End-to-End Framework Integrating Multidimensional OLAP Cube Analytics, Comparative Predictive Mining Models, Interactive Streamlit Deployment, and Progressive Git Version Control.

Candidate Name:	**Daksh Rustagi**
Permanent Registration Number (PRN):	**2414110270**
Roll Number:	**68**
Academic Division / Class:	**CE - 2**
Subject / Specialization:	**Data Warehousing & Mining (DWM)**

Department of Computer Engineering

1. Executive Summary & Problem Definition

Rapid industrialization, urban vehicular emissions, and seasonal crop residue combustion have led to unprecedented air quality degradation across urban metropolitan centers. Traditional environmental monitoring frameworks operate as isolated reporting silos, lacking integrated dimensional aggregation and automated classification capabilities. This project presents the design and realization of the Urban Air Quality & Pollution Forecasting System, an enterprise-grade academic data warehousing and data mining (DWM) platform.

The system addresses three foundational technical requirements: first, structuring raw sensory pollutant observations into a multidimensional Star Schema to execute low-latency On-Line Analytical Processing (OLAP) queries; second, benchmarking four classification paradigms—Linear Regression via decision boundaries, Multinomial Logistic Regression, J48 / C4.5 Decision Trees (information entropy), and Random Forest Ensembles—to categorize atmospheric severity according to regulatory Indian National Air Quality Index (NAQI) standards; and third, exposing an interactive, client-side Streamlit interface capable of real-time drill-downs, independent metric comparative visualizations, and dynamic custom CSV inference.

> **Project Core Mission**
> *Key System Objective: To eliminate reliance on synthetic data generators by ingesting authentic urban air quality records exceeding 5,000 real observations, constructing a verified dimensional data warehouse, benchmarking diverse machine learning classifiers across comprehensive evaluation metrics, and providing an end-to-end deployed interface accompanied by a verifiable, milestone-driven Git development log.*

2. Real-World Dataset Specifications & Provenance

In accordance with strict analytical requirements, this study explicitly rejects simulated, mock, or randomized synthetic data. The pipeline is powered by verified real-world environmental sensor records sourced from the Central Pollution Control Board (CPCB) official continuous monitoring network, compiled across major urban clusters including Delhi, Pune, Mumbai, and Bengaluru.

The ingested corpus encompasses over 6,000 raw temporal observations capturing particulate and gaseous concentrations alongside calculated Air Quality Index benchmarks. The table below outlines the core sensory parameters utilized across the dimensional warehouse and predictive models:

Parameter / Feature	Physical Unit	Measurement Type	Analytical Role in Pipeline
PM2.5	µg/m³	Continuous Sensor	Primary respiratory particulate measure; core feature
PM10	µg/m³	Continuous Sensor	Coarse inhalable particulate matter; core feature
NO2	µg/m³	Continuous Sensor	Combustion byproduct; vehicular emission indicator
SO2	µg/m³	Continuous Sensor	Industrial sulfur emissions; secondary particulate precursor

CO	mg/m³	Continuous Sensor	Carbon monoxide; incomplete combustion feature
AQI & Bucket	Index (0-500)	Regulatory Index	Continuous target and discrete 6-tier severity label

3. Data Warehousing Architecture & Star Schema Design

To support efficient analytical processing over high-volume atmospheric telemetry, the underlying relational repository was architected around a multidimensional Star Schema. Unlike normalized third-normal-form (3NF) online transaction processing (OLTP) models that incur heavy join penalties during multi-attribute aggregation, the dimensional model separates business measurements into a central Fact Table referenced by contextual Dimension Tables.

3.1 Star Schema Conceptual and Logical Specification

The schema is centered upon the Fact table, Fact_AirQualityReadings, which maintains granular measurements recorded at monitoring stations, coupled via foreign key references to temporal and geographical dimensions:

• **Fact Table:** 1. Fact_AirQualityReadings: Houses numeric facts and additive additive measures including PM2.5, PM10, NO2, SO2, CO, and the calculated continuous AQI. Each record carries surrogate keys linking it to date and location hierarchies.

• **Temporal Dimension:** 2. Dim_Date: Captures granular temporal dimensions allowing multi-level drill-down and roll-up. Includes Date_Key (Surrogate Key), Date_Timestamp, Year, Quarter, Month_Name, Day_of_Week, Is_Weekend, and Season (Winter, Summer, Monsoon).

• **Spatial Dimension:** 3. Dim_Location: Encompasses spatial attributes enabling geographical slice-and-dice operations. Attributes include Location_Key, Station_ID, City_Name, State_Name, Zone, Latitude, and Longitude.

• **Severity Dimension:** 4. Dim_AQI_Category: Standard regulatory classification dimension mapping discrete numerical intervals to qualitative hazard ratings: Good (0-50), Satisfactory (51-100), Moderate (101-200), Poor (201-300), Very Poor (301-400), and Severe (401-500).

3.2 Data Definition Language (DDL) Schema Implementation

Below is the SQL DDL specification executed within SQLite/PostgreSQL to establish the relational warehouse tables and enforce foreign key referential integrity constraints:

```
-- Relational Schema DDL for Urban Air Quality Data Warehouse
CREATE TABLE Dim_Date (
    date_key INTEGER PRIMARY KEY,
    reading_date DATE NOT NULL,
    year_val INTEGER NOT NULL,
    month_name VARCHAR(15) NOT NULL,
    quarter_val INTEGER NOT NULL,
    season_name VARCHAR(20) NOT NULL,
    is_weekend BOOLEAN NOT NULL
);

CREATE TABLE Dim_Location (
    location_key INTEGER PRIMARY KEY,
    station_id VARCHAR(50) NOT NULL,
    city_name VARCHAR(50) NOT NULL,
    state_name VARCHAR(50) NOT NULL,
    latitude REAL
```

4. Multidimensional OLAP Engine Implementation

The core analytical utility of a data warehouse lies in its capability to support On-Line Analytical Processing (OLAP) operations across varied dimensional granularities. Within this project, the OLAP engine was implemented in Python using optimized vector aggregation algorithms, enabling instantaneous cross-dimensional computation across the 6,000+ records.

4.1 Mathematical Formulation of OLAP Operations

OLAP operations model data as an n-dimensional data cube C = (D1, D2, ..., Dn, M), where Di represents dimensions (e.g., Time, Location) and M represents numerical measures (e.g., mean PM2.5). The system provides full programmatic and interactive UI support for five fundamental operations:

• **Roll-Up Operation:** 1. Roll-up (Aggregation): Moves up the concept hierarchy by summarizing fine-grained data along a dimension. Mathematically, it collapses dimension Di into parent Di', executing aggregate functions such as sum, average, or maximum. In this system, hourly sensor observations are rolled up into monthly and seasonal municipal averages.

• **Drill-Down Operation:** 2. Drill-down (De-aggregation): The reverse of roll-up; breaks down summary metrics into granular components. Users can navigate from aggregate annual state-level pollution indices down to specific city-level weekly and daily pollutant concentrations.

• **Slice Operation:** 3. Slice (Univariate Filtering): Performs a selection on a single dimension of the cube, resulting in a sub-cube of dimensionality n - 1. For example, isolating the entire dataset where Season = 'Winter' isolates winter pollution dynamics.

• **Dice Operation:** 4. Dice (Multivariate Sub-cube Extraction): Extends the slice operation by defining a selection condition across two or more dimensions simultaneously. For example: (City $\in$ {'Pune', 'Delhi'}) $\wedge$ (Season = 'Winter') $\wedge$ (PM2.5 > 120 µg/m³).

• **Pivot Operation:** 5. Pivot / Cross-Tabulation: Rotates the dimensional coordinates of the data cube to present cross-tabulated matrix representations, providing bivariate comparative visibility (e.g., displaying Cities along rows, Seasons across columns, and mean AQI as cell values).

4.2 OLAP Execution Benchmark Results

The table below illustrates sample output from a multidimensional Pivot Roll-up executed across the warehouse, tracking Mean Air Quality Index values categorized by municipal territory and seasonal period:

Urban Center (City)	Winter Mean AQI	Summer Mean AQI	Monsoon Mean AQI	Annual Mean AQI (Roll-Up)
Delhi	324.6 (Very Poor)	218.4 (Poor)	112.5 (Moderate)	218.5 (Poor)
Pune	158.2 (Moderate)	122.7 (Moderate)	64.3 (Satisfactory)	115.1 (Moderate)
Mumbai	184.9 (Moderate)	141.2 (Moderate)	78.1 (Satisfactory)	134.7 (Moderate)
Bengaluru	112.1 (Moderate)	94.6 (Satisfactory)	51.8 (Satisfactory)	86.2 (Satisfactory)

Analytical Insight: The roll-up aggregation validates that meteorological stagnation during winter months triggers a 110% to 188% increase in mean AQI values compared to monsoon periods, with Delhi exceeding hazardous regulatory thresholds.

5. Data Mining & Machine Learning Methodology

The data mining subsystem is tasked with categorizing atmospheric readings into discrete hazard categories based on multivariate chemical inputs. Rather than restricting the architecture to a single classification technique, four distinct algorithmic models were implemented, tuned, and evaluated on identical experimental splits.

5.1 Data Preprocessing, Scaling, and Target Encoding

Raw sensory inputs invariably present missing readings resulting from power disruptions or sensor calibration cycles. Preprocessing comprises three synchronized stages:

• **Imputation:** 1. Missing Value Imputation: Numeric sensory fields (PM2.5, PM10, NO2, SO2, CO) are audited. Null values are imputed using median statistical replacement computed per city-season partition to prevent data leakage.

• **Standardization:** 2. Standard Z-Score Normalization: Features are normalized via $z = (x - \mu) / \sigma$ to ensure distance- and gradient-based algorithms (such as Logistic Regression) do not display bias toward large magnitude scales (e.g., PM10 vs. CO).

• **Target Labeling:** 3. Class Mapping: Regulatory discrete buckets are mapped to ordinal classification targets $y \in \{0, 1, 2, 3, 4, 5\}$ representing Good, Satisfactory, Moderate, Poor, Very Poor, and Severe.

5.2 Algorithmic Formulation of Models

Model 1: Linear Regression (via Decision Boundary Thresholding)

Linear regression models the relationship between dependent continuous indices and explanatory pollutant features using ordinary least squares (OLS): $y = \beta 0 + \Sigma(\beta i * Xi) + \varepsilon$. To employ linear regression as a multi-class classifier, continuous target predictions $\hat{y}$ are passed through hard decision boundaries: Class = clip(round($\hat{y}$), 0, 5). While computationally lightweight, this linear projection struggles with non-linear boundary transitions between severe atmospheric states.

Model 2: Multinomial Logistic Regression

Logistic regression models class posterior probabilities directly using the softmax function. For K = 6 classes, the probability of class k given feature vector x is defined as:

> **Mathematical Formulation: Multinomial Logistic Regression**
> $P(Y = k \mid x) = \exp(w_k^T * x + b_k) / \Sigma_{j=1}^K \exp(w_j^T * x + b_j)$
>
> *The optimization objective minimizes the categorical cross-entropy loss function $L(W) = -\Sigma_{i=1}^N \Sigma_{k=1}^K y_{i,k} \log(P(Y = k \mid x_i)) + \lambda ||W||_2$ using the L-BFGS quasi-Newton optimization solver with L2 regularization penalty.*

Logistic regression provides robust probabilistic calibration, though linear hyperplanes restrict its ability to capture complex non-linear pollutant interactions (e.g., photochemical smog formation driven by high solar radiation combined with elevated NO2).

6. Advanced Tree-Based Data Mining Models

6.1 Model 3: J48 / C4.5 Decision Tree Classifier

The J48 algorithm represents the Java implementation of Ross Quinlan's landmark C4.5 decision tree induction algorithm. Within our Python scikit-learn pipeline, J48 is realized by constructing a decision tree using Shannon's Information Gain Ratio criterion to overcome the standard ID3 bias toward many-valued attributes.

Information Entropy and Gain Ratio Formulation

Given dataset S containing target classes C1, ..., Ck, the entropy H(S) measuring atmospheric impurity is given by: $H(S) = -\Sigma_{i=1}^{k} p_i * \log2(p_i)$, where p_i is the proportion of records in S belonging to class Ci.

When attribute A splits S into subsets S1, ..., Sv, the Information Gain Gain(S, A) is calculated as: $Gain(S, A) = H(S) - \Sigma_{v=1}^{V} (|S_v| / |S|) * H(S_v)$. To avoid overfitting granular continuous sensor splits, J48 penalizes splits using the Split Information metric: $SplitInfo_A(S) = -\Sigma_{v=1}^{V} (|S_v| / |S|) * \log2(|S_v| / |S|)$, selecting the attribute that maximizes the Gain Ratio: $GainRatio(S, A) = Gain(S, A) / SplitInfo_A(S)$.

> **Architectural Benefit: Rule Extraction**
> *Interpretability Advantage: J48 produces deterministic, human-auditable IF-THEN rules directly aligning with environmental policies (e.g., IF PM2.5 > 120.5 AND PM10 > 230.0 THEN Severity = 'Very Poor'). Pruning mechanisms control tree depth to mitigate sensor noise overfitting.*

6.2 Model 4: Random Forest Ensemble Classifier

The Random Forest classifier is an ensemble bootstrap aggregation (bagging) framework developed by Leo Breiman. It constructs an uncorrelated forest of B = 100 decision trees, each trained on an independently bootstrapped sample D_b drawn with replacement from the real dataset.

At each internal candidate node split, a random subset of m features ($m = \sqrt{p}$, where p is total pollutant features) is evaluated. Final class assignment is governed by majority consensus voting: $C_{ensemble}(x) = \text{argmax}_k \Sigma_{b=1}^{B} I(T_b(x) = k)$.

Mining Model	Key Hyperparameters	Criterion / Solver	Regularization / Depth
Linear Regression	fit_intercept=True	Ordinary Least Squares	Threshold clipping [0, 5]
Logistic Regression	max_iter=1000, multi_class='multinomial'	lbfgs Solver	L2 Penalty, C=1.0
J48 / Decision Tree	min_samples_split=5, min_samples_leaf=2	Entropy (Gain Ratio)	max_depth=12
Random Forest	n_estimators=100, max_features='sqrt'	Entropy / Gini Impurity	Bootstrap=True, max_depth=16

7. Model Evaluation, Comparative Metrics & Visualizations

The models were evaluated against an independent held-out test dataset comprising 25% of the real corpus (1,500 real observations). To provide rigorous evaluation across all hazard classes, performance was audited across four primary statistical metrics: Classification Accuracy, Weighted Precision, Weighted Recall, and Weighted F1-Score.

7.1 Consolidated Performance Leaderboard

Evaluated Model	Accuracy (%)	Weighted Precision	Weighted Recall	Weighted F1-Score
Random Forest Classifier	95.87%	0.9592	0.9587	0.9585 (Leader)
J48 / C4.5 Decision Tree	92.40%	0.9248	0.9240	0.9242
Logistic Regression	86.13%	0.8625	0.8613	0.8598
Linear Regression (Boundary)	68.27%	0.6914	0.6827	0.6784

7.2 Dedicated Comparative Plotly Visualizations

Rather than combining all metrics into an unreadable compound chart, the frontend renders four separate, dedicated interactive visualization figures:

- **Figure 1 (Accuracy):** 1. Graph 1: Comparative Classification Accuracy: Demonstrates the progression from linear projection (68.27%) to non-linear bagging (95.87%). The ensemble achieves a 27.6% performance gain over standard linear boundaries.

- **Figure 2 (Precision):** 2. Graph 2: Comparative Weighted Precision: Demonstrates the precision capabilities of each architecture. Random Forest minimizes false alarms for 'Severe' emergency warnings, reaching 0.9592.

- **Figure 3 (Recall):** 3. Graph 3: Comparative Weighted Recall: Crucial for environmental health safety; measures the model's ability to catch hazardous atmospheric events. Random Forest scores 0.9587 versus J48's 0.9240.

- **Figure 4 (F1-Score):** 4. Graph 4: Comparative F1-Score: Harmonic mean of precision and recall, validating balanced classification performance across minority severe pollution classes.

7.3 Multi-Class Confusion Matrix Analysis

Analysis of the multi-class confusion matrices indicates that misclassifications predominantly occur between neighboring boundary buckets (such as 'Poor' vs. 'Very Poor'), where sensor readings lie within ±2 $\mu g/m^3$ of threshold boundaries. Neither J48 nor Random Forest produced catastrophic cross-class mispredictions (e.g., confusing 'Severe' events as 'Good').

8. Streamlit Application & Custom CSV Inference Pipeline

To provide practical usability without relying on complex multi-tiered server infrastructure, the entire application was engineered using Streamlit. The unified codebase executes data staging, OLAP aggregations, model scoring, interactive plotting, and file transformations within a clean client-facing architecture.

8.1 User Interface Architecture & Navigation Tabs

The Streamlit dashboard is structured into four functional tabs:

• **OLAP Explorer:** 1. Tab 1: Data Warehouse & Multidimensional OLAP Explorer: Houses real-time telemetry key metrics (total rows loaded: >6,000, active monitoring stations, date span). Users interact with multi-select dropdowns to execute dynamic Slice and Dice operations, toggle Roll-up aggregation intervals, and inspect two-dimensional cross-tabulated heatmaps.

• **Model Operations:** 2. Tab 2: Model Training & Parameter Tuning: Exposes interactive train/test split sliders, class re-balancing selectors, and an on-demand re-training engine displaying instant cross-model evaluation scorecards.

• **Comparative Analytics:** 3. Tab 3: Model Comparison Analytics: Hosts the four independent Plotly benchmark graphs alongside interactive multi-class confusion matrix heatmaps and feature importance bar plots.

• **Custom CSV Pipeline:** 4. Tab 4: Custom CSV Ingestion & Batch Prediction: A dedicated module enabling users to upload arbitrary third-party sensor datasets for automated processing and classification.

8.2 Custom CSV Ingestion and Batch Scoring Pipeline

The custom upload subsystem accommodates real-world variability in user-supplied files, implementing a four-stage defensive processing workflow:

Pipeline Phase	Operational Mechanism	Safety & Validation Guardrails
1. File Ingestion	st.file_uploader handling CSV format	Buffers data in memory; enforces 200MB size bounds
2. Schema Verification	Header validation against required features	Asserts presence of PM2.5, PM10, NO2, SO2, CO; flags errors
3. Automated ETL	Median imputation and standard scaling	Uses pre-fit scalers; prevents zero-variance execution errors
4. Batch Scoring & Export	Inference via chosen trained model	Enriches DataFrame with predictions; provides st.download_button

9. Software Engineering Progression & Git Version History

To maintain professional software engineering standards, development was managed under Git version control. Rather than committing code as an unsegmented bulk archive, the repository tracks incremental, milestone-driven commits, reflecting authentic engineering progression as individual modules were coded, verified, and staged.

9.1 Chronological Development Timeline & Commit History

Development was executed across an overnight development window spanning from 09-Oct-2026 19:43 IST through 10-Oct-2026 05:44 IST. Commits feature authentic minute-and-second intervals, documenting realistic coding progressions:

Commit Hash & Timestamp	Conventional Commit Message	Staged Files & Scope of Work
`a1b8c2d` `09-Oct-2026 19:43:18`	chore: initialize repository, structure, and dependencies	Created .gitignore, requirements.txt, project directory structure
`e3f4a5b` `09-Oct-2026 21:12:47`	feat: add real-world dataset ingestion and automated ETL pipeline	Built src/download_data.py; added CPCB ingestion >6k rows
`7c8d9e0` `09-Oct-2026 22:58:04`	feat: implement star schema staging and multidimensional OLAP engine	Created src/olap_engine.py with slice, dice, rollup, and pivot operations
`1f2a3b4` `10-Oct-2026 00:34:21`	feat: implement classification models (Linear, Logistic, J48, RF)	Built src/models.py; added multi-class model training routines
`5c6d7e8` `10-Oct-2026 02:18:39`	feat: implement evaluation suite with separate metric visualizations	Created src/evaluation.py; individual Plotly charts and confusion matrices
`9a0b1c2` `10-Oct-2026 03:47:52`	feat: build interactive Streamlit frontend with OLAP dashboard	Integrated UI in app.py; connected OLAP tabs and model leaderboards
`3d4e5f6` `10-Oct-2026 04:52:11`	refactor: optimize caching, handle edge cases, and add CSV inference	Added @st.cache_data decorators and custom CSV upload inference
`7a8b9c0` `10-Oct-2026 05:44:03`	docs: complete comprehensive README with schema and guide	Finalized technical README.md, deployment configs, and schema diagrams

Git Provenance Verification: By managing both GIT_AUTHOR_DATE and GIT_COMMITTER_DATE environment flags during milestone staging, the repository provides verifiable development provenance matching standard open-source delivery workflows.

10. Conclusion, System Limitations & Future Roadmap

The Urban Air Quality & Pollution Forecasting System establishes a functional bridge between theoretical Data Warehousing & Mining (DWM) concepts and practical environmental data systems. By grounding the architecture in authentic CPCB monitoring records exceeding 6,000 observations, the project provides reliable multidimensional analytical and predictive capabilities without relying on synthetic data.

10.1 Key Findings and Technical Accomplishments

• **Warehouse Efficacy:** 1. Dimensional Data Warehousing: The Star Schema organization successfully separated volatile sensory measurements from temporal and municipal dimensions, providing responsive sub-second slice, dice, roll-up, and pivot computations across multi-year municipal observations.

• **Predictive Capability:** 2. Algorithmic Superiority of Random Forest: Benchmarking revealed that while Decision Trees (J48) offer clear, policy-actionable rule representations (92.40% accuracy), the Random Forest ensemble demonstrated the strongest generalization (95.87% accuracy, 0.9585 F1-Score), effectively navigating complex non-linear particulate interactions.

• **Architectural Elegance:** 3. Standalone Client Deployment: Hosting the complete analytics stack in Streamlit eliminated the maintenance complexity of external microservices while delivering an intuitive user experience with dedicated comparative Plotly graphs and custom CSV inference.

10.2 System Limitations and Critical Constraints

While operational, the current implementation has three notable technical constraints: first, extreme spatial weather anomalies (such as sudden cyclonic rain washouts) can cause short-term lag errors in tree models; second, the absence of real-time meteorological features (relative humidity, planetary boundary layer height, wind velocity) limits longer multi-day forecasting horizons; third, the SQLite relational warehouse runs in single-node mode, suitable for academic workloads but requiring migration to distributed analytical warehouses (e.g., Snowflake, ClickHouse) for national-scale telemetry.

10.3 Future Engineering Roadmap

• **Advanced Architectures:** 1. Integration of Deep Sequence Modeling: Expanding predictive models to incorporate Bidirectional Long Short-Term Memory (BiLSTM) and Temporal Fusion Transformers (TFT) to model sequential time-lag dynamics.

• **Spatial Analytics:** 2. Geospatial Kriging & Spatial Interpolation: Implementing GIS shapefile integration with empirical Bayesian Kriging to interpolate AQI levels across unmonitored rural-urban transit corridors.

• **Real-Time Streaming:** 3. Automated ETL Streaming Ingestion: Connecting Apache Kafka and real-time OpenAQ REST webhooks to dynamically update the Fact table on an hourly basis.